Prashant Gupta PRN: 250840325052

Hive Assignment 2

Part A: Table Creation and Data Loading

1. Create a managed table named airports with appropriate columns and data types based on the dataset provided.

```
hive (easy1)> create table airport(airport_id int, name string,city string,country string,iata string,icao string,latitude doub
le,longitude double,altitude int,timezone double,dst string,tz string)row format delimited fields terminated by ',' stored as t
extfile;
```

2. Create a managed table named airlines with the correct schema and data types.

```
Time taken: 0.057 seconds
hive (easy1)> create table airlines(airline_id int,name string,alias string,iata string,icao string,callsign string,country str
ing,active string)row format delimited fields terminated by ',' stored as textfile;
OK
Time taken: 0.044 seconds
```

3. Create a managed table named routes using the dataset structure provided.

```
hive (easy1)> create table routes(airline_iata string,airline_id int,src_airport_iata string, src_airport_id int, dest_airport_
iata string, dest_airport_id int, codeshare string, stops int, equipment string)row format delimited fields terminated by ',' s
tored as textfile;
OK
```

4. Load data from local files (airports.txt, airlines.txt, routes.txt) into their respective tables.

```
hive (easy1)> load data local inpath '/home/cdacotnv40018/airports_mod.dat' into table airport;
Loading data to table easy1.airport
OK
Time taken: 4.036 seconds
hive (easy1)>
```

```
hive (easy1)> load data local inpath '/home/cdacotnv40018/Final_airlines' into table airlines;
Loading data to table easy1.airlines
OK
Time taken: 4.041 seconds
```

```
hive (easy1)> load data local inpath '/home/cdacotnv40018/routes.dat' into table routes;
Loading data to table easy1.routes
OK
Time taken: 4.103 seconds
```

5. Create an external table for airports that references data stored in a specific HDFS directory. Explain the difference in behavior when dropping the internal vs. external table.

```
hive (easy1)> create external table ext_airport(airport_id int, name string,city string,country string,iata string,icao string,
latitude double,longitude double,altitude int,timezone double,dst string,tz string)row format delimited fields terminated by ',
' stored as textfile location '/user/cdacotnv40018/a1';
OK
Time taken: 0.049 seconds
hive (easy1)> show tables;
OK
```

1. Dropping an Internal (Managed) Table

When you drop an internal (or managed) table, the system assumes full ownership and control over both the table definition and its data.

- Metadata Deletion: The table's metadata (schema, column definitions, etc.) is deleted from the metastore (e.g., Hive Metastore, Unity Catalog).
- Data Deletion: The actual data files associated with the table are deleted from the underlying file system (e.g., HDFS, S3).

In essence, dropping an internal table is a complete cleanup—it removes the table's logical definition and its physical data.

> Use Case: You should use an internal table when you want the data platform (like Hive) to manage the entire lifecycle of the data, and you are comfortable with the data being deleted permanently when the table is dropped (or moved to a trash directory, depending on configuration).

2. Dropping an External Table

When you drop an external table, the system manages only the table definition (metadata), while the data itself is expected to be managed by an external process or user.

- Metadata Deletion: The table's metadata (schema, column definitions, etc.) is deleted from the metastore.
- Data Deletion: The actual data files at the specified `LOCATION` are NOT deleted; they remain untouched in the underlying file system.

Dropping an external table is a metadata-only operation—it simply removes the pointer to the data, allowing the data to be safely reused or managed outside of the table definition.

B: Joins and Querying

6. Display all distinct countries that have at least one active airline operating.

   select country,count(case when active="Y" then 1 end)as c from airlines group by country having c>=1;

```
 S.A.   1
Afghanistan     4
Albania 2
Australia       26
Belarus 2
Benin   1
Bolivia 2
Brazil  23
British Virgin Islands  1
Cambodia        5
Colombia        9
Comoros 1
Honduras        1
Indonesia       17
Israel  6
Jamaica 1
Jordan  2
Kazakhstan      6
Kenya   3
Madagascar      1
Marshall Islands        1
```

7. List all airports along with their corresponding airline names by performing an appropriate join.
   select a.airport_id as airport_id,a.name as air_name,a1.name from airport a join airlines a1 on a1.iata=a.iata limit 30;

```
4163    Bergrestaurant  FTI Fluggesellschaft
4163    Bergrestaurant  Go2Sky
4163    Bergrestaurant  Grand Cru Airlines
4163    Bergrestaurant  Transilvania
4163    Bergrestaurant  CARICOM AIRWAYS (BARBADOS) INC.
4163    Bergrestaurant  Envoy Air
4163    Bergrestaurant  Skyline Ulasim Ticaret A.S.
4163    Bergrestaurant  Yeti Airlines
4163    Bergrestaurant  Vintage Props and Jets
4163    Bergrestaurant  Tomsk-Avia
4163    Bergrestaurant  BBN-Airways
4163    Bergrestaurant  Royal Flight
4163    Bergrestaurant  Simrik Airlines
4163    Bergrestaurant  Key Air
4163    Bergrestaurant  Russkie Krylya
4163    Bergrestaurant  FLYJET
4163    Bergrestaurant  Comfort Express Virtual Charters
4163    Bergrestaurant  Comfort Express Virtual Charters Albany
4163    Bergrestaurant  Transair
4163    Bergrestaurant  Sunrise Airways
4163    Bergrestaurant  Canaryfly
4163    Bergrestaurant  ViznAir
4163    Bergrestaurant  Golden Myanmar Airlines
4163    Bergrestaurant  Nürnberger Flugdienst
4163    Bergrestaurant  Flyme (VP)
4163    Bergrestaurant  Eastern Atlantic Charters
```

8. Find all routes that operate between airports located in the same country.

```
hive (easy1)> select r.* from routes r join airport src on src.airport_id=r.src_airport_id join airport dest on dest.airport_id=r.dest_airport_id where src.country=dest
.country limit 10;
```

```
Stage-Stage-1: Map: 2  Reduce: 4   Cumulative CPU: 13.71 sec   HDFS Read: 31
Stage-Stage-2: Map: 3  Reduce: 4   Cumulative CPU: 15.81 sec   HDFS Read: 44
Total MapReduce CPU Time Spent: 29 seconds 520 msec
OK
6L      NULL    YPC     4244    YEV     54              0       DHT
6L      NULL    YHI     68      YEV     54              0       DHT
HW      2748    YGH     4239    YEV     54              0       DHT
6L      NULL    ZFM     206     YEV     54              0       DHT
6L      NULL    YUB     145     YEV     54              0       DHT
5T      1623    YVQ     155     YEV     54              0       73M 733
7F      2354    YVQ     155     YEV     54              0       73M
6L      NULL    YSY     139     YEV     54              0       DHT
Q9      11838   YGV     7255    YPN     106             0       PAG
Q9      11838   YZV     202     YPN     106             0       PAG
Time taken: 45.527 seconds, Fetched: 10 row(s)
hive (easy1)>
```

9. Retrieve details of airlines that have more than 10 routes originating from Indian airports.

```
Select a.airline_id,a.name,count(*) from airlines a join routes r
on r.airline_id=a.airline_id where a.country=”India” group by
a.airline_id,a.name limit 10;
```

```
Total MapReduce CPU Time Spent: 21 seconds 200 msec
OK
a.airline_id    a.name  _c2
569     Air India Express       76
4375    Spicejet        196
218     Air India Limited       393
2850    IndiGo Airlines 227
241     Air Sahara      180
2575    Go Air  77
3000    Jet Airways     292
Time taken: 47.387 seconds, Fetched: 7 row(s)
hive (easy1)>
```

10. Display the top 5 countries with the highest number of airports.

```
select country,count(airport_id)as c from airport group by country
order by c desc limit 5;
```

```
United States   1697
Canada  435
Germany 321
Australia       263
Russia  249
```

C: ORC and Parquet Storage Formats

11. Create a copy of the routes table in ORC format.

```
CREATE TABLE routes_orc STORED AS ORC AS SELECT * FROM routes;
```

12. Create another copy of the routes table in Parquet format.

```
hive (easy1)> create table routes_parquet stored as parquet as select * from routes;
```

13. Compare the storage size of the ORC and Parquet tables using HDFS commands.

```
cdacotnv40018@ip-172-31-15-133:~$ hdfs dfs -du -h /user/hive/warehouse/easy1.db/routes_orc
341.1 K  1023.3 K  /user/hive/warehouse/easy1.db/routes_orc/000000_0
cdacotnv40018@ip-172-31-15-133:~$ hdfs dfs -du -h /user/hive/warehouse/easy1.db/routes_parquet
560.5 K  1.6 M  /user/hive/warehouse/easy1.db/routes_parquet/000000_0
cdacotnv40018@ip-172-31-15-133:~$ hdfs dfs -du -h /user/hive/warehouse/easy1.db/routes
2.3 M  6.8 M  /user/hive/warehouse/easy1.db/routes/routes.dat
cdacotnv40018@ip-172-31-15-133:~$
```

14. Run a simple count query on each table and compare performance.

```
hive (easy1)> select count(*) from routes_orc;
OK
_c0
67663
Time taken: 3.379 seconds, Fetched: 1 row(s)
hive (easy1)> select count(*) from routes_parquet;
OK
_c0
67663
Time taken: 2.349 seconds, Fetched: 1 row(s)
hive (easy1)>
```

15. Explain the difference in how data is stored in TextFile, ORC, and Parquet formats.

1. Text File (e.g., CSV, JSON):

- Row-Oriented: Data is stored row by row, meaning all the values for a single record are stored together contiguously.
- Human-Readable: Text files are generally human-readable, making them easy to inspect and debug.
- Text files typically do not enforce a schema, making them flexible but potentially prone to data quality issues.
- Compression is often applied at the file level, not optimized for individual data types, leading to less efficient storage. Querying requires scanning entire rows, even if only specific columns are needed.

2. ORC (Optimized Row Columnar):

- Hybrid Row-Columnar: ORC stores data in a columnar format within independent "stripes" (groups of rows). Each stripe has its own index, row data, and footer.
- Optimized for Hive: ORC was designed with Hive in mind and offers features like ACID transactions for Hive tables.

- High Compression & Efficient Reads: Columnar storage allows for highly efficient compression due to data type similarity within columns. It supports predicate pushdown and vectorized reads, significantly improving query performance, especially for analytical workloads.
- Schema Evolution: ORC offers reasonable support for schema evolution, allowing for additions of new columns.

3. Parquet:

- Columnar: Parquet is a purely columnar storage format, meaning data for each column is stored separately and contiguously.
- Language-Independent: Parquet is designed to be language-independent and is widely used across various big data processing frameworks like Spark, Impala, and Presto.
- Columnar storage enables advanced compression techniques tailored to each column's data type. It excels in read-heavy analytical queries by allowing selective column reads and predicate pushdown, reducing I/O.
- Schema Evolution: Parquet has robust support for schema evolution, including adding, removing, or reordering columns.
- Parquet is particularly well-suited for handling complex, nested data structures.

D: ACID and Transactional Tables

16. Create a transactional table for airlines_txn that supports INSERT, UPDATE, and DELETE operations.

```
hive (easy1)> set hive.support.concurrency=true;
hive (easy1)> set hive.txn.manager=org.apache.hadoop.hive.ql.lockmgr.DbTxnManager;
hive (easy1)> set hive.enforce.bucketing=true;
hive (easy1)> CREATE TABLE airlines_txn (
            >
            > airline_id int,
            >
            > name string,
            >
            > alias string,
            >
            > iata string,
            >
            > icao string,
            >
            > callsign string,
            >
            > country string,
            >
            > active string
            >
            > )
            >
            > CLUSTERED BY (airline_id) INTO 4 BUCKETS
            >
            > STORED AS ORC
            >
            > TBLPROPERTIES ('transactional'='true');
OK
Time taken: 0.334 seconds
```

17. Insert data from the existing airlines table into airlines_txn.

```
hive (easy1)> insert into airlines_txn select * from airlines;
Query ID = cdacotnv40018_20251113084406_17f21a9c-5819-4e94-8d63-d6c490b95c3d
```

18. Update the active status of an airline to 'N' for a specific airline_id.

```
hive (easy1)> select * from airlines_txn where active="N" limit 5;
OK
airlines_txn.airline_id airlines_txn.name       airlines_txn.alias      airlines_txn.iata
  airlines_txn.icao       airlines_txn.callsign   airlines_txn.country    airlines_txn.active
3036    Jetcorp NULL            UEJ     JETCORP United States   N
3040    Jetfly Airlines NULL            JFL     LINEFLYER       Austria N
56      Air Cess        NULL            ACS             Liberia N
4552    Spurling Aviation       NULL            ASL     AIR SEATTLE     United States   N
3044    Jetnova de Aviacion Ejecutiva   NULL            JNV     JETNOVA Spain   N
Time taken: 2.168 seconds, Fetched: 5 row(s)
```

```
hive (easy1)> select * from airlines_txn where airline_id=3036;
OK
airlines_txn.airline_id airlines_txn.name       airlines_txn.alias      airlines_txn.iata
  airlines_txn.icao       airlines_txn.callsign   airlines_txn.country    airlines_txn.active
3036    Jetcorp NULL            UEJ     JETCORP United States   Y
Time taken: 2.107 seconds, Fetched: 1 row(s)
hive (easy1)>
```

19.Delete records of airlines that are inactive.

```
hive (easy1)> delete from airlines_txn where active="N";
```

```
hive (easy1)> select count(*) from airlines_txn where active="N";
```

```
OK
_c0
0
```

19. Perform a major compaction on the transactional table and verify the results.

```
hive (easy1)> Alter table airlines_txn compact 'major';
Compaction enqueued with id 1898685
OK
Time taken: 0.039 seconds
```

```
1898730 easy1   airlines_txn     ---     MAJOR   failed  master.cloudloka.com-30 1763024665000
  7000    job_1762934466274_1119
```

E: Hive Functions and UDFs

21. Using built-in Hive functions, display the total number of airlines by country.

```
> select country ,count(*) from airlines_txn group by country;
```

```
country _c1
NULL    2
        2
 S.A.   1
Afghanistan     4
Albania 2
Australia       26
Belarus 2
Benin   1
Bolivia 2
Brazil  23
British Virgin Islands  1
Cambodia        5
Colombia        9
Comoros 1
Honduras        1
Indonesia       17
Israel  6
Jamaica 1
Jordan  2
Kazakhstan      6
Kenya   3
Madagascar      1
Marshall Islands        1
Moldova 2
Mongolia        3
Montenegro      1
Morocco 5
Nepal   6
Netherlands     9
```

22. Use a string function to extract the first three characters of the airport name for all airports.

```
hive (easy1)> select substr(name,0,3) from airport limit 10;
OK
_c0
Gor
Mad
Mou
Nad
Por
Wew
Nar
Nuu
Son
Thu
Time taken: 2.216 seconds, Fetched: 10 row(s)
hive (easy1)>
```

23. Use a conditional expression to classify airports based on altitude as 'High', 'Medium', or 'Low'.

```
hive (easy1)> select altitude,case when altitude between -1266 and 7000 then 'LOW' when altitude between 7001 and 12000 then "MEDIUM" else "HIGH" end as Category from a
irport limit 10;
OK
altitude        category
5282    LOW
20      LOW
5388    LOW
239     LOW
146     LOW
19      LOW
112     LOW
283     LOW
165     LOW
251     LOW
Time taken: 1.919 seconds, Fetched: 10 row(s)
```

24. Create a Python-based UDF using the TRANSFORM clause to convert airline names to uppercase.

```
add file /home/cdacotnv40018/scripts/airline_upper.py;
```

select transform (name) Using 'python airline_upper.py' as (upper_name) from airlines;

```
PORTUGALIA
PORTUGUESE AIR FORCE
PORTUGUESE ARMY
PORTUGUESE NAVY
POTOMAC AIR
POTOSINA DEL AIRE
POWELL AIR
PRAIRIE FLYING SERVICE
PRATT AND WHITNEY CANADA
PRECISION AIR
PRECISION AIRLINES
PREMIAIR
PREMIAIR AVIATION SERVICES
PREMIAIR FLIYNG CLUB
PREMIUM AIR SHUTTLE
PREMIUM AVIATION
PRESIDENCE DU FASO
PRESIDENCIA DE LA REPUBLICA DE GUINEA ECUATORIAL
PRESIDENT AIRLINES
PRESIDENTIAL AVIATION
PRIESTER AVIATION
PRIMAIR
PRIMARIS AIRLINES
PRIMAS COURIER
PRIMAVIA LIMITED
PRIME AIR
PRIME AVIATION
PRINCE EDWARD AIR
PRINCELY JETS
PRINCESS AIR
PRINCETON AVIATION CORPORATION
PRIORITY AIR CHARTER
```

25. Create another Python-based UDF to mask the first three letters of the airport name for privacy purposes.

```
            > add file /home/cdacotnv40018/scripts/masked_airport.py
            > ;
Added resources: [/home/cdacotnv40018/scripts/masked_airport.py]
hive (easy1)>
```

```
hive (easy1)> select transform(name) using 'python3 masked_airport.py'as masked from airport;
Query ID = cdacotnv40018_20251114044421_4bd15736-3ccd-47a6-91ba-ffb794ad89d5
Total jobs = 1

***inikon International Airport
***sai
***l Street Heliport
***bilaran
***lissat
***igiannguit
***iaat
*** Sant Joan
***win Intl
***at Thani
***an Intl
***ofredo P
***ini North Seaplane Base
***keetna
***kija Heliport
***ed-New Haven Airport
***eville Regional Airport
***dmont Triad
***ux Falls
***rs Rock
***chester Regional Airport
***les Muni
***ang
***isville International Airport
***rlottesville-Albemarle
***noke Regional
***e Grass
***nsville Regional
***uquerque International Sunport
***latin Field
***lings Logan International Ai
```

Part F: Analytical and Scenario-Based Queries

26. Find all airlines that operate direct flights (stops = 0) between two given airports.

```
           > set hive.cli.print.header=true;
hive (easy1)> select a.name as Airline,r.stops as stops from airlines a join routes r on a.airline_id=r.airline_id where r.stop
s=0 limit 5;
```

```
Total MapReduce CPU Time Spent: 12 seconds 550 msec
OK
airline stops
40-Mile Air     0
40-Mile Air     0
40-Mile Air     0
40-Mile Air     0
Aigle Azur      0
Time taken: 23.225 seconds, Fetched: 5 row(s)
hive (easy1)>
```

27. Retrieve the total number of routes per airline and order the result by the highest number of routes.
    Select airline_id,count(*) as routes from airlines group by airline_id order by routes desc limit 10;

```
OK
airline_id      routes
38      1
34      1
19834   1
26      1
22      1
18      1
14      1
10      1
19830   1
2       1
Time taken: 556.752 seconds, Fetched: 10 row(s)
hive (easy1)>
```

28. Identify countries where no active airlines are currently operating.

```
Time taken: 0.031 seconds, Fetched: 8 row(s)
hive (easy1)> select country,active from airlines where active='N' limit 5;
OK
country active
United States   N
United Kingdom  N
Russia  N
Russia  N
Russia  N
Time taken: 2.738 seconds, Fetched: 5 row(s)
hive (easy1)>
```

29. Display the number of airports per timezone.

```
hive (easy1)>
            > select tz as Timezone, count(*) as No_of_Airports from airport group by tz;
Query ID = cdacotnv40018_20251114050639_297f131d-a83d-401e-b5c5-97263ecf55c5

timezone        no_of_airports
NULL    92
Africa/Algiers  44
Africa/Bamako   8
Africa/Bissau   2
Africa/Blantyre 8
Africa/Bujumbura        1
Africa/Gaborone 25
Africa/Harare   16
Africa/Kampala  9
Africa/Lome     2
Africa/Luanda   25
Africa/Niamey   7
America/Anguilla        1
America/Antigua 2
America/Belize  12
America/Cayman  3
America/Coral_Harbour   3
America/Godthab 22
America/Guadeloupe      6
America/Guatemala       10
America/Havana  29
America/La_Paz  27
America/Martinique      4
America/Mexico_City     65
America/Nassau  37
America/Puerto_Rico     15
America/Sao_Paulo       97
America/St_Kitts        3
```

30. Using a join across all three tables, display the airline name, source airport, destination airport, and number of stops for flights operated by airlines based in India.

```
hive (easy1)> With c1 as( Select airline_id,src.name as Source,dest.name as Destination,r.stops as Stops from routes r join air
port src on src.airport_id=r.src_airport_id join airport dest on dest.airport_id=r.dest_airport_id where src.country="India") S
elect a1.name,c1.Source,c1.Destination,c1.Stops from airlines a1 join c1 on c1.airline_id=a1.airline_id limit 10;
No Stats for easy1@airlines, Columns: name, airline_id
No Stats for easy1@routes, Columns: dest_airport_id, airline_id, stops, src_airport_id
```

```
OK
a1.name cl.source       cl.destination  cl.stops
Go Air  Jammu   Chhatrapati Shivaji Intl        0
Go Air  Jaipur  Chhatrapati Shivaji Intl        0
Go Air  Jammu   Srinagar        0
Go Air  Jammu   Indira Gandhi Intl      0
Go Air  Leh     Indira Gandhi Intl      0
Go Air  Birsa Munda     Chhatrapati Shivaji Intl        0
Go Air  Goa     Bangalore       0
Go Air  Goa     Indira Gandhi Intl      0
Go Air  Birsa Munda     Indira Gandhi Intl      0
Go Air  Goa     Chhatrapati Shivaji Intl        0
Time taken: 66.868 seconds, Fetched: 10 row(s)
hive (easy1)>
```

Part G: Conceptual and Descriptive Questions

31. Explain the purpose of bucketing in Hive and its importance in ACID tables.

Bucketing provides three main analytical and performance benefits:

1. Efficient Sampling: Bucketing makes it easy to select a representative sample of the data. Since rows are distributed deterministically based on a bucket column's hash, you can query a specific fraction of the data by targeting a few bucket files without scanning the entire dataset.
    - *Example:* To get a 10% sample of customer data, you can query a specific bucket if the table is bucketed into 10 buckets.
2. Map-Side Joins: If two tables are bucketed by the same column and have the same number of buckets, Hive can execute a Map-Side Join.[3] Instead of performing a large shuffle (Redistribute Join) across the network, the engine only needs to join corresponding bucket files (Bucket 1 of Table A joins only with Bucket 1 of Table B). This greatly reduces data transfer and improves join performance.[4]
3. Data Organization and Uniformity: Bucketing provides a structured, hash-based distribution of data, ensuring that the records are relatively evenly spread across the bucket files, which helps with workload balancing during processing.[5]

Importance in Hive ACID Tables

For Full ACID (Update, Delete, Merge) tables in Hive, bucketing is historically and functionally critical because it is fundamental to Hive's ability to locate and manage rows for transactional consistency.

1. Row Identification: Bucketing is essential for creating the unique Row ID that Hive's transaction manager uses to track every record. Without bucketing, the system loses a key piece of information needed for unambiguous row identification.
2. Efficient Compaction: ACID tables generate many small delta files when updates or deletes occur. To prevent query slowdowns, Hive runs compaction jobs to merge these deltas with the base files.

   Bucketing ensures that the data is partitioned into distinct bucket files. Compaction can then be performed in parallel and independently on each bucket file, dramatically

speeding up the overall cleanup process and reducing the risk of a single, massive compaction job failing due to resource limits.

3. Locking Granularity: Bucketing can help reduce lock contention by allowing concurrent transactions to potentially operate on different buckets of the same table simultaneously.

While some modern versions of Hive (3+) can auto-assign a default single bucket if bucketing is not explicitly specified, explicit bucketing is the required design pattern for performant and reliable ACID functionality.

32. Differentiate between managed and external tables in Hive with respect to data storage and deletion behavior.

Feature	Managed (Internal) Table	External Table
Data Ownership	Hive owns the data. The Hive Metastore manages both the table metadata (schema) and the actual data files.	Data is owned externally. Hive only manages the table metadata (schema). The data files belong to the underlying storage (HDFS, S3, etc.).
Data Storage Location	Data files are stored in Hive's default warehouse directory, typically under a path like `/user/hive/warehouse/database_name.db/table_name`.	Data files are stored in a user-specified location using the `LOCATION` clause during table creation.
Deletion Behavior	When you execute `DROP TABLE`, Hive deletes both the metadata from the Metastore AND the underlying data files from HDFS/storage.	When you execute `DROP TABLE`, Hive deletes only the metadata from the Metastore. The underlying data files remain intact in their original location.

Data Loading	Data is moved into the table's dedicated warehouse directory.	Hive simply creates a link to the existing data; files are not moved or copied.

33. Describe the Map-Side and Reduce-Side join process in Hive.

Map-Side and Reduce-Side joins are two primary methods for performing joins in Hive, each leveraging the MapReduce framework differently for performance and scalability.

Map-Side Join (or Map-Join)

A **Map-Side Join** is an optimization technique used when one of the tables involved in the join is **significantly smaller** than the other(s). It's generally much faster than a Reduce-Side Join because it avoids the shuffle and sort phases.

Process

1. **Distribution/Broadcast:** The small table is loaded entirely into memory on the driver (Hive client/resource manager).
2. **Broadcast to Mappers:** This small table is then **broadcast** (sent) to all Map tasks running on the cluster. Each mapper receives a full copy of the small table and stores it in a hash table (or similar in-memory structure).
3. **Joining:** When a mapper processes a record from the large table (the primary input), it looks up the corresponding record in the in-memory small table using the join key.
4. **Output:** The joined records are immediately output by the mapper.

Key Characteristics

- **Avoids Shuffle/Sort:** The costly shuffle and sort phase required for grouping and joining keys in the reducers is completely bypassed.
- **Performance:** Significantly faster for joining large and small datasets.
- **Memory Constraint:** The total size of the small table(s) **must fit entirely into the memory** of the Map task's heap. If it's too large, the join will fail or fall back to a Reduce-Side join.
- **Hive Trigger:** Can be explicitly enabled using the hint /*+ MAPJOIN(small_table) */ or automatically triggered when the table size is below the threshold set by the property hive.auto.convert.join.noconditionaltask.size (default is typically around 10MB to 25MB).

Reduce-Side Join

A **Reduce-Side Join** is the standard, more general-purpose method for joining tables in Hive and is used when tables are of **comparable or large sizes**.

Process

1. **Map Phase:**
    - **Input:** Both tables are read independently by their respective Mappers.
    - **Transformation:** Each mapper processes records from its input table and emits a key-value pair where:
        - **Key:** The **join key**
        - **Value:** The non-join columns of the record
2. **Shuffle and Sort Phase:**
    - The MapReduce framework takes the key-value pairs from all mappers.
    - All records with the **same join key** are guaranteed to be sent to the **same Reducer**.
    - The values are then sorted by key.
3. **Reduce Phase:**
    - The Reducer receives all records grouped by the join key.
    - For a specific join key, the reducer iterates through the list of values
    - The reducer performs the actual join logic and emits the final joined record.

34. Explain how Hive determines whether to use a map-side join automatically.

Hive automatically determines whether to use a map-side join (also called an Auto Map Join or Broadcast Join) primarily by checking the size of the tables involved in a join.A map-side join is an optimization where the smaller table is loaded into memory on all mapper nodes, allowing the join to be performed entirely in the map phase, which skips the costly shuffle and sort phase that requires reducers. This significantly speeds up query execution.

hive.auto.convert.join: This parameter must be set to true (which is often the default in modern Hive versions). When true, Hive's optimizer will look for opportunities to convert standard joins into map-side joins.

- *Default Value:* true

hive.mapjoin.smalltable.filesize: This parameter defines the **maximum size** a table can be to be considered "small" and eligible to be broadcasted and cached for a map-side join. Hive checks the file size of the tables involved.

- *Default Value:* Typically **25 MB**. If the size of one table in an equi-join is less than this value, Hive will attempt to convert it into a map-side join, broadcasting the smaller table.

35. Discuss the internal storage structure of ORC and Parquet files and how it helps improve query performance.

Feature	ORC (Optimized Row Columnar)	Parquet
Main Data Unit	**Stripes** (large, independent blocks)	**Row Groups** (main unit of horizontal partition)
Indexing	Built-in lightweight indexes with **Min/Max statistics** stored within stripes.	Statistics (Min/Max) stored at **Row Group** and **Page** levels in the file footer.
Splittability	Highly splittable, often using the stripe as the unit for parallel reading.	Highly splittable, often using the row group as the unit.
Primary Ecosystem	Initially developed for **Hive/MapReduce** (often performs better with Hive).	Developed by Twitter, widely adopted by **Impala/Spark** (often performs better with Spark/Impala).
Key Optimization	Focuses on combining data storage (columns within stripes) for efficient HDFS reads.	Focuses on sophisticated encoding and compression at the **Page** level for maximum space savings.